

Informe tecnico - Primera entrega parcial del TPI (Food Store)

Fabri - UTN. Base food_store_copia (nunca toque la dev). PostgreSQL 18 + DBeaver en Manual, con backup antes como dice el protocolo.

1. Que se implemento en cada unidad

Arranque del TP1 con el modelo: diagrama ER de 5 entidades donde detecte la N:M entre pedido y producto, que resolvi con la intermedia pedido_producto. Lo pase a relacional con las FK y sus acciones justificadas (RESTRICT para no borrar clientes con facturas ni productos historicos, CASCADE para las lineas si se anula un ticket), y lo normalize hasta 3FN con las dependencias documentadas. El DDL quedo en Food Store.sql con lo propio del motor: ENUM forma_pago_enum, IDENTITY, TIMESTAMPTZ, UNIQUE en email, CHECKs de precios/stock/cantidad y los flags de baja logica con DEFAULT.

En la Unidad 2 meti las reglas R1 (precio_unitario <= precio_lista) y R2 (fecha no futura) como funciones + triggers en PL/pgSQL, y reproduci 3 escenarios de concurrencia en dos sesiones (no repetible, fantasma y bloqueo) probando READ COMMITTED contra REPEATABLE READ. Todo con el protocolo de siempre: copia, transaccion Manual y respaldo.

La Unidad 3 fue la saga de optimizacion: carga masiva de 200k en TP3, plan de indexado con EXPLAIN ANALYZE en TP3/TP4/TP5 (con mis documentadas no efectivas incluidas), vistas y la materializada de facturacion en TP5, mas el ranking con RANK() y los nunca vendidos con NOT EXISTS en TP4. Para cerrar el parcial me quedaban 3 huecos del checklist y los tape en food-store-parcial/: la consulta con HAVING, el procedimiento con CALL y la demo de borrado logico.

2. Como se probó el funcionamiento

Como en toda la cursada: nada se ejecuta sin spec de Kiro leído ni sin entender linea por linea lo que genero OpenCode, todo sobre food_store_copia y todo reversible. Aca me lleve la leccion del parcial: el DDL tambien es transaccional en Manual. Cree la vista, la vi andar en una pestana (50002) y en la otra me decia que no existia (42P01); habia quedado sin commitear y un Revertir la borro. De ahi la regla que dejo escrita: el DDL estructural se commitea, solo los experimentos terminan en ROLLBACK. Las equivalencias, como siempre, con EXCEPT en ambos sentidos.

3. Que resultados se obtuvieron (checklist de 9)

Obj	Evidencia	Resultado
1. ER	diagrama TP1	5 entidades, N:M detectada
2. Relacional	Food Store.sql	intermedia + FKs justificadas
3. 3FN/BCNF	desarrollo TP1	DF documentadas
4. DDL	schema + indices	ENUM/IDENTITY/TZ/UNIQUE
5. DML	queries + H1	JOIN, GROUP BY, HAVING, EXISTS, RANK
6. Vistas/func/proc	views, triggers, B0	3 vistas + MV + 2 func + 1 proc
7. Reglas	restricciones.sql	CHECK/UNIQUE y triggers R1-R2
8. Transacciones	informe_concurrencia	3 anomalias reproducidas
9. Borrado logico	B1/B2/B3	CALL: 50001 y 0 filas; ROLLBACK: 50002

El HAVING me sorprendio: esperaba ~10 filas (carga pareja) y dio 6, o sea que filtra de verdad (30M/40M/50M pasan, el resto no). La demo de borrado quedo redonda: CALL sp_baja_producto(3), la vista pasa de 50002 a 50001 y el producto 3 da 0 filas; ROLLBACK y vuelve todo a 50002. El procedimiento ademas tira excepcion si el producto no existe o ya esta dado de baja.

4. Consultas optimizadas y diferencias antes/despues

Consulta	Antes	Despues	X
TP3 Q3 precio	Seq Scan 3.394 ms	Bitmap 1.636 ms	2.07x
TP3 Q1	Bitmap 2.101 ms	Bitmap nuevo 1.430 ms	1.46x
TP4 Q-A / Q-B	Hash 430 / 152.9 ms	sigue en Hash	no efectivas
TP5 Q1	Seq Scan 43.185 ms	Index Scan 0.031 ms	1393x
TP5 Q2	Bitmap+Filter 9.581 ms	Index Cond 0.248 ms	38.6x
TP5 Q3	Seq Scan 19.779 ms	sigue Seq 14.065 ms	no efectiva
MV facturacion	agregado ~430 ms	precargada 0.038 ms	~11300x

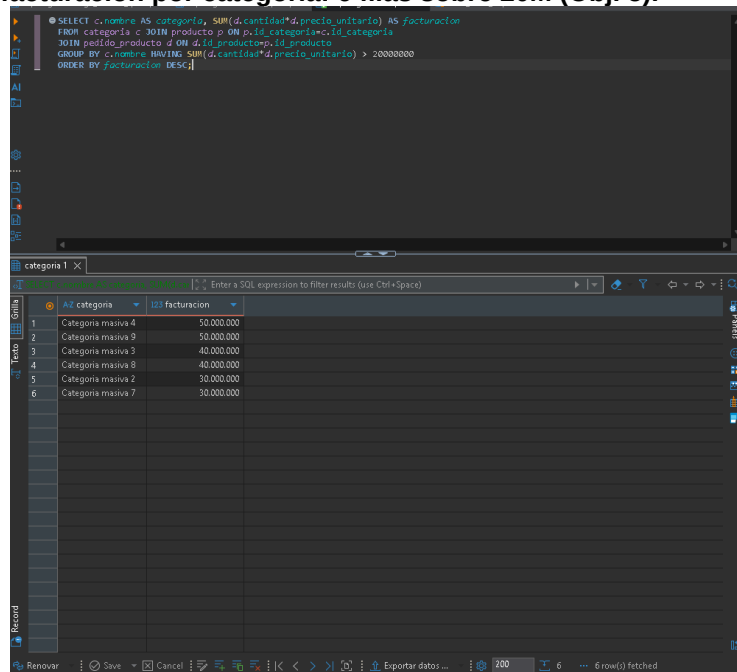
La lectura es la de siempre: el covering de TP4 no alcanza porque para agregar todo hay que leer todo (Hash en memoria le gana a 200k lookups), y en TP5 el compuesto/parcial si alcanza porque el filtro recorta al 1-5%. Cuando recorta al 40% (Q3) el planificador vuelve a Seq Scan, y esta bien. El indice no es bueno o malo: es selectivo o no.

5. Uso de IA

Kiro para especificar y OpenCode para generar, como pide la catedra; no use otras IA. Acepte los indices compuestos/parciales, las vistas, la MV y el procedimiento. Descarte con justificacion: el indice solo en forma_pago (cardinalidad 1), los covering de TP4 (no efectivos) y las explicaciones de planes que confundian costo con tiempo. Cada TP tiene su bitacora DUJA.

Anexo - Capturas de cierre de gaps

H1 - HAVING de facturacion por categoria: 6 filas sobre 20M (Obj. 5).



```
SELECT c.nombre AS categoria, SUM(d.cantidad*d.precio_unitario) AS facturacion
FROM categoria c JOIN producto p ON p.id_categoria=c.id_categoria
JOIN pedido_producto d ON d.id_producto=p.id_producto
GROUP BY c.nombre HAVING SUM(d.cantidad*d.precio_unitario) > 20000000
ORDER BY facturacion DESC;
```

	All categoria	id+ facturacion
1	Categoria masiva 4	50.000.000
2	Categoria masiva 9	50.000.000
3	Categoria masiva 3	40.000.000
4	Categoria masiva 8	40.000.000
5	Categoria masiva 2	30.000.000
6	Categoria masiva 7	30.000.000

B0 - CREATE PROCEDURE sp_baja_producto en 0.066 s (Obj. 6).

The screenshot shows the PostgreSQL Script Editor with a SQL script to create a procedure. The script is as follows:

```
CREATE OR REPLACE PROCEDURE sp_baja_producto(p_id BIGINT) LANGUAGE plpgsql AS
$$ BEGIN UPDATE producto SET activo=FALSE WHERE id_producto=p_id AND activo=TRUE;
IF NOT FOUND THEN RAISE EXCEPTION 'producto % inexistente o ya dado de baja', p_id; END IF; END; $$
```

The Statistics window (Estadísticas 1) shows the following details:

Name	Value
Updated Rows	0
Execute time	0.066s
Start time	Wed Sep 23 10:44:39 GMT-03:00 2026
Finish time	Wed Sep 23 10:44:39 GMT-03:00 2026
Query	CREATE OR REPLACE PROCEDURE sp_baja_producto(p_id BIGINT) LANGUAGE plpgsql AS \$\$ BEGIN UPDATE producto SET activo=FALSE WHERE id_producto=p_id AND activo=TRUE; IF NOT FOUND THEN RAISE EXCEPTION 'producto % inexistente o ya dado de baja', p_id; END IF; END; \$\$

B1 - CALL sp_baja_producto(3): la vista pasa a 50001 (Obj. 9).

The screenshot shows the PostgreSQL Script Editor with a SQL script to call the procedure. The script is as follows:

```
BEGIN;  
CALL sp_baja_producto(3);  
SELECT * FROM v_productos_vigentes WHERE id_producto=3;  
SELECT COUNT(*) FROM v_productos_vigentes;  
ROLLBACK;
```

The Statistics window (Estadísticas 2) shows the following details:

Count	Value
1	50,001

B2 - El producto 3 ya no aparece en la vista: 0 filas (Obj. 9).

Script: C:\Users\Fabri\OneDrive\Documents\Default Project\sql\carga_masiva_tp3.sql

Connection: postgres

Type: PostgreSQL

URL: jdbc:postgresql://localhost:5432/postgres

Database: food_store_copia

Schema: public

id_producto	nombre_producto	precio_lista	stock	nombre_categoria
123	A2 nombre_producto	123	123	A2 nombre_categoria

GMT-03:00 es Editable Inserción inteligente 5:10:145

B3 - Tras el ROLLBACK: 50002 restaurado, base intacta (Obj. 8 y 9).

postgres> cargo main v103-10 | postgres> index v103-10 | postgres> Script 6: X
 SELECT COUNT(*) FROM v_productos_vigentes; -- 50002

v_productos_vigentes 1		Estadísticas 1	
	103 count		
1	50,002		

GMT-03:00 | es | Editable | Inserción inteligente | 1:1:0 | Sel: 0 | 0